

Accessing Network Storage *with* Network File System

There are a number of steps that need to be taken before Network File System (NFS) allows clients access to data. This article describes those steps and also guides the reader on some other aspects of NFS.



Network File System or NFS is an Internet standard protocol used by Linux, UNIX and similar operating systems as their native network file system. It is an open standard under active extension, which supports native Linux permissions and file system features. Red Hat Enterprise Linux 7 supports NFSv4 (version 4 for the protocol) by default, and falls back automatically to NFSv3 and NFSv2 if that is not available. NFSv4 uses the TCP protocol to communicate with the server, while older versions of NFS may use either TCP or UDP.

NFS servers export share directories and NFS clients mount an exported share to a local mount point (directory). The local mount point must exist. NFS shares can be mounted in a number of ways:

- Manual mounting using the *mount* command
- Automatic mounting at boot time using */etc/fstab*
- Mounting on demand through a process known as automounting

Securing file access on NFS shares

NFS servers secure access to files using a number of methods such as *none*, *sys*, *krb5*, *krb5i* and *krb5p*. The NFS server can choose to offer a single or multiple methods for each exported share. NFS clients must connect to the exported share using

one of the methods mandated for that share, specified as a *mount option sec= method*.

Methods used to secure access to files

- **None:** This gives anonymous access to the files, and writes to the server (if allowed) are allocated UID and GID of *nfsnobody*.
- **Sys:** This gives file access based on standard Linux file permissions for UID and GID values. If not specified, this is the default.
- **Krb5:** Clients must prove their identity using Kerberos, and then standard Linux file permissions apply.
- **Krb5i** adds a cryptographically strong guarantee that the data in each request has not been tampered with.
- **Krb5p** adds encryption to all requests between the client and the server, preventing data exposure on the network. This has a performance impact.

Important: Kerberos options will require, as a minimum, a */etc/krb5.* keytab and additional authentication configuration. The */etc/krb5.* keytab will normally be provided by the authentication or security administrator. Request a keytab that includes either a host principal, NFS principal, or (ideally) both.